Audience: LEWG, SG14, WG21
Document number: D0447R10
Date: 2019-12-10
Project: Introduction of std::colony to the standard library
Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

Introduction of std::colony to the standard library

Table of Contents

I. Introduction

The purpose of a container in the standard library cannot be to provide the most optimal solution for all scenarios. Inevitably in fields such as high-performance trading or gaming, the optimal solution within critical loops will be a custom-made one that fits that scenario perfectly. However, outside of the most critical of hot paths, there is a wide range of application for more generalised solutions.

Colony is a formalisation, extension and optimization of what is typically known as a 'bucket array' container in game programming circles; similar structures exist in various incarnations across the high-performance computing, high performance trading, physics simulation, robotics, server/client application and particle simulation fields (see: https://groups.google.com/a/isocpp.org/forum/#!topic/sg14/1iWHyVnsLBQ).

The concept of a bucket array is: you have multiple memory blocks of elements, and a boolean token for each element which denotes whether or not that element is 'active' or 'erased'. If it is 'erased', it is skipped over during iteration. When all elements in a block are erased, the block is removed, so that iteration does not lose performance by having to skip empty blocks. If an insertion occurs when all the blocks are full, a new memory block is allocated.

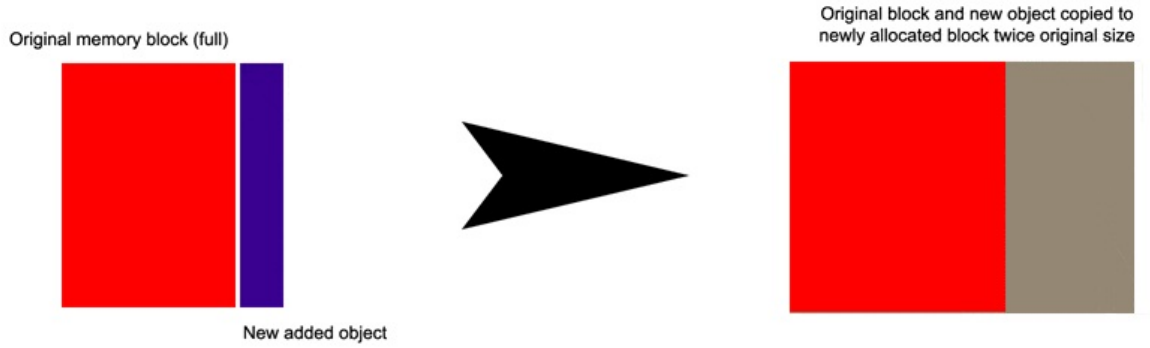
The advantages of this structure are as follows: because a skipfield is used, no reallocation of elements is necessary upon erasure. Because the structure uses multiple memory blocks, insertions to a full container also do not trigger reallocations. This means that element memory locations stay stable and pointers/references/indexes stay valid regardless of erasure/insertion. This is highly desirable, for example, in game programming because there are usually multiple elements in different containers which need to reference each other during gameplay and elements are being inserted or erased in real time.

Problematic aspects of a typical bucket array are that they tend to have a fixed memory block size, do not re-use memory locations from erased elements, and utilize a boolean skipfield. The fixed block size (as opposed to block sizes with a

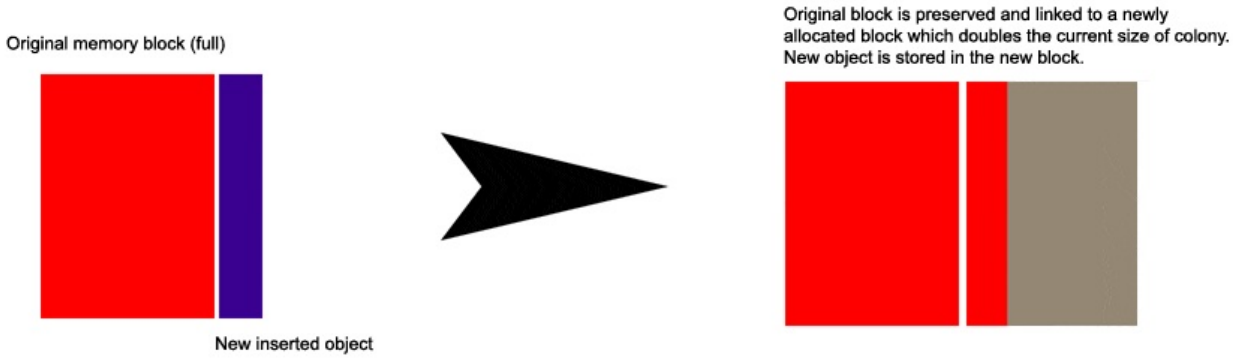
growth factor) and lack of erased-element re-use leads to far more allocations/deallocations than is necessary. Given that allocation is a costly operation in most operating systems, this becomes important in performance-critical environments. The boolean skipfield makes iteration time complexity undefined, as there is no way of knowing ahead of time how many erased elements occur between any two erased elements. This can create variable latency during iteration. It also requires branching code, which may cause issues on processors with deep pipelines and poor branch-prediction failure performance.

A colony uses a non-boolean, largely non-branching method for skipping *runs* of erased elements, which allows for O(1) amortised iteration time complexity and more-predictable iteration performance than a bucket array. It also utilizes a growth factor for memory blocks and reuses erased element locations upon insertion, which leads to fewer allocations/reallocations. Because it reuses erased element memory space, the exact location of insertion is undefined, unless no erasures have occurred or an equal number of erasures and insertions have occurred (in which case the insertion location is the back of the container). The container is therefore considered unordered but sortable. Lastly, because there is no way of predicting in advance where erasures ('skips') may occur during iteration, an O(1) time complexity [ ] operator is impossible and the container is bidirectional, but not random-access.

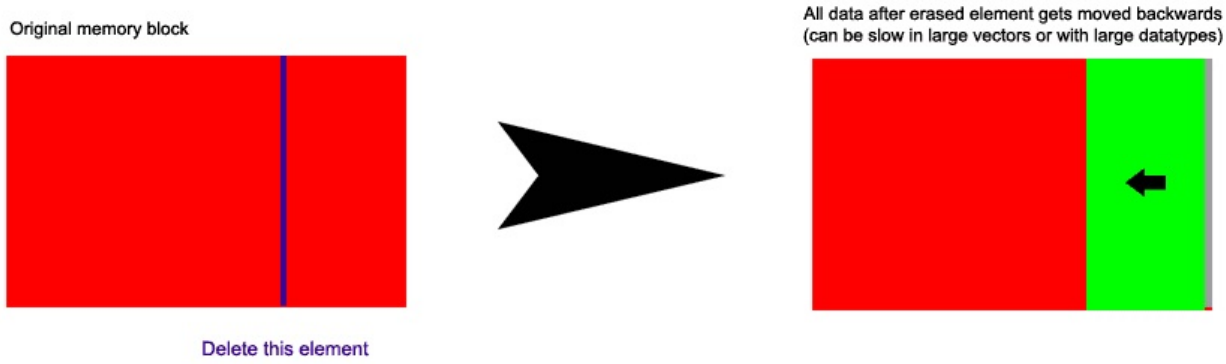
Vector:



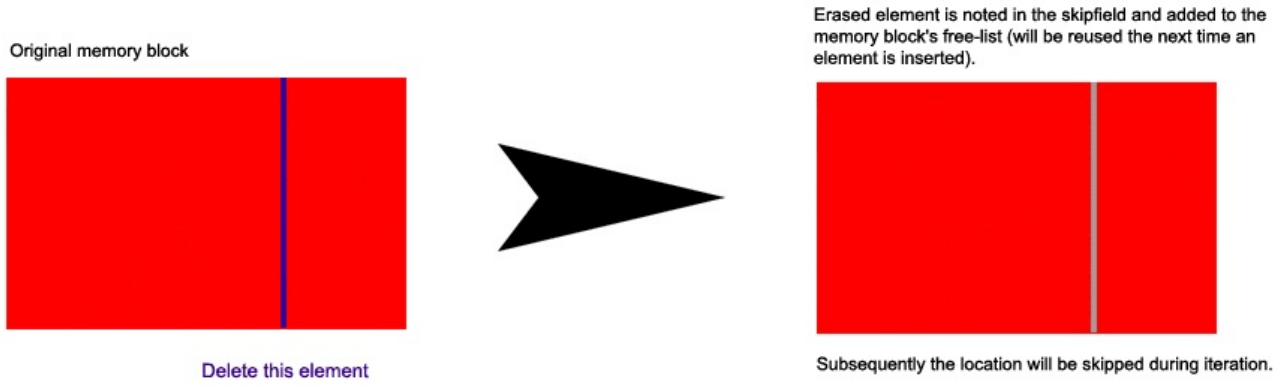
Colony:



Vector:



Colony:



There are two patterns for accessing stored elements in a colony: the first is to iterate over the container and process each element (or skip some elements using the advance/prev/next/iterator ++/-- functions). The second is to store the iterator returned by the insert() function (or a pointer derived from the iterator) in some other structure and access the inserted element in that way.

II. Motivation and Scope

Note: Throughout this document I will use the term 'link' to denote any form of referencing between elements whether it be via iterators/pointers/indexes/references/ids/etc.

There are situations where data is heavily interlinked, iterated over frequently, and changing often. An example is the typical video game engine. Most games will have a central generic 'entity' or 'actor' class, regardless of their overall schema (an entity class does not imply an ECS). Entity/actor objects tend to be 'has a'-style objects rather than 'is a'-style objects, which link to, rather than contain, shared resources like sprites, sounds and so on. Those shared resources are

usually located in separate containers/arrays so that they can re-used by multiple entities. Entities are in turn referenced by other structures within a game engine, such as quadtrees/octrees, level structures, and so on.

Entities may be erased at any time (for example, a wall gets destroyed and no longer is required to be processed by the game's engine, so is erased) and new entities inserted (for example, a new enemy is spawned). While this is all happening the links between entities, resources and superstructures such as levels and quadtrees, must stay valid in order for the game to run. The order of the entities and resources themselves within the containers is, in the context of a game, typically unimportant, so an unordered container is okay.

Unfortunately the container with the best iteration performance in the standard library, vector[1], loses pointer validity to elements within it upon insertion, and pointer/index validity upon erasure. This tends to lead to sophisticated and often restrictive workarounds when developers attempt to utilize vector or similar containers under the above circumstances.

std::list and the like are not suitable due to their poor locality, which leads to poor cache performance during iteration. This is however an ideal situation for a container such as colony, which has a high degree of locality. Even though that locality can be punctuated by gaps from erased elements, it still works out better in terms of iteration performance[1] than every existing standard library container other than deque/vector, regardless of the ratio of erased to non-erased elements.

Some more specific requirements for containers in the context of game development are listed in the appendix.

As another example, particle simulation (weather, physics etcetera) often involves large clusters of particles which interact with external objects and each other. The particles each have individual properties (spin, momentum, direction etc) and are being created and destroyed continuously. Therefore the order of the particles is unimportant, what is important is the speed of erasure and insertion. No current standard library container has both strong insertion and non-back erasure speed, so again this is a good match for colony.

Reports from other fields suggest that, because most developers aren't aware of containers such as this, they often end up using solutions which are sub-par for iteration such as std::map and std::list in order to preserve pointer validity, when most of their processing work is actually iteration-based. So, introducing this container would both create a convenient solution to these situations, as well as increasing awareness of better-performing approaches in general. It will also ease communication across fields, as opposed to the current scenario where each field uses a similar container but each has a different name for it.

III. Impact On the Standard

This is a pure library addition, no changes are necessary to the standard asides from the introduction of the colony container.
A reference implementation of colony is available for download and use here.

IV. Design Decisions

The three core aspects of a colony from an abstract perspective are:

1. A collection of element memory blocks + metadata, to prevent reallocation during insertion (as opposed to a single memory block)
2. A non-boolean skipfield, to enable O(1) skipping of erased elements during iteration (as opposed to reallocating subsequent elements during erasure)
3. An erased-element location recording mechanism, to enable the re-using of memory from erased elements during subsequent insertions

Each memory block houses multiple elements. The metadata about each block may or may not be allocated with the blocks themselves (could be contained in a separate structure). This metadata might include, for example, the number of erased elements within each block and the block's capacity - which would allow the container to know when the block is empty and needs to be removed. A non-boolean skipfield is required in order to skip over erased elements during iteration while maintaining O(1) amortised iteration time complexity. Finally, a mechanism for keeping track of elements which have been erased must be present, so that those memory locations can be reused upon subsequent element insertions.

The following aspects of a colony must be implementation-defined in order to allow for variance and possible performance improvement, or conformance to future epochs of C++ in implementations:

- the skipfield structure
- skipfield modification time complexity

- erasure-recording mechanism
- element memory block metadata
- iterator structure
- memory block growth factor
- time complexity of advance()/next()/prev()

However their implementation *is* significantly constrained by the requirements of the container (lack of reallocation and stable pointers to non-erased elements regardless of erasures/insertions, etcetera).

In terms of the reference implementation, the specific structure and mechanisms have changed many times over the course of development, however the interface to the container and its time complexity guarantees have remained largely unchanged (with the exception of the time complexity for updating skipfield nodes). So it is reasonably likely that regardless of specific implementation, it will be possible to maintain this general specification without obviating future improvements in implementation, so long as time complexity guarantees for updating skipfields are left implementation-defined.

Below I will explain the reference implementation's approach in terms of the three core aspects described above, along with some descriptions of implementation alternatives.

1. Collection of element memory blocks + metadata

In the reference implementation this is essentially a doubly-linked list of 'group' structs containing (a) memory blocks, (b) memory block metadata and (c) skipfields. The memory blocks and skipfields have a growth factor of 2 from one group to the next. The metadata includes information necessary for an iterator to iterate over colony elements, such as the last insertion point within the memory block, and other information useful to specific functions, such as the total number of non-erased elements in the node. This approach keeps the operation of freeing empty memory blocks from the colony container at O(1) time complexity. Further information is available here.

An alternative implementation could be to use a vector of pointers to dynamically-allocated memory blocks + skipfields in a single struct, with a separate vector of memory block metadata structs. Such an approach would have some advantages in terms of increasing the locality for metadata during iteration, but would create reallocation costs when memory blocks + their skipfields and metadata were removed upon becoming empty.

A vector of memory blocks, as opposed to a vector of pointers to memory blocks, would not work as it would (a) disallow a growth factor in the memory blocks and (b) invalidate pointers to elements in subsequent blocks when a memory block became empty of elements and was therefore removed from the vector. In short it would negate all of a colony's beneficial aspects.

2. A non-boolean skipfield which allows for O(1) traversal from each non-erased element to the next

The reference implementation currently uses a skipfield pattern called the *Low complexity jump-counting pattern* (formerly under working title 'bentley pattern', current version of paper in-progress). This effectively encodes the run-length of sequences of consecutive erased elements, into a skipfield, which allows for O(1) time complexity during iteration. Since there is no branching involved in iterating over the skipfield aside from end-of-block checks, it can be less problematic computationally than a boolean skipfield (which has to branch for every skipfield read) in terms of CPUs which don't handle branching or branch-prediction failure efficiently.

The pattern stores and modifies the run-lengths during insertion and erasure with O(1) time complexity. It has a lot of similarities to the High complexity jump-counting pattern, which was the pattern previously used by the reference implementation. Using the High complexity jump-counting pattern is an alternative, though the skipfield update time complexity guarantees for that pattern are effectively undefined, or between O(1) and O(skipfield length) for each insertion/erasure. In practice those updates result in one memcpy operation which resolves to a single block-copy operation, but it is still a little slower than the Low complexity jump-counting pattern regardless. The skipfield pattern you use will also typically have an effect on the type of memory-reuse mechanism you can utilize.

A pure boolean skipfield is not usable because it makes iteration time complexity undefined - it could for example result in thousands of branching statements + skipfield reads for a single ++ operation in the case of many consecutive erased elements. In the high-performance fields for which this container was initially designed, this brings with it unacceptable latency. However another strategy using a combination of a jump-counting *and* boolean skipfield, which saves memory at the expense of computational efficiency, is possible as follows:

1. Instead of storing the data for the low complexity jump-counting pattern in it's own skipfield, have a boolean bitfield indicating which elements are erased. Store the jump-counting data in the erased element's memory space instead.
2. When iterating, check whether the element is erased or not using the boolean bitfield, if not, do nothing. If it is erased, read the jump value from the erased element's memory space and skip forward the appropriate number of nodes, both in the element

memory block and the boolean bitfield.

This approach has the advantage of still performing O(1) iterations from one non-erased element to the next, unlike a pure boolean skipfield approach, but compared to a pure-jump-counting approach introduces 3 additional costs per iteration via (1) a branch operation when checking the bitfield, (2) an additional read (of the erased element's memory space) and (3) a bitmasking operation to read the bit. But it reduces the memory overhead of the skipfield to 1 bit per-element.

3. Erased-element location recording mechanism

There are two valid approaches here; both involve per-memory-block free lists, utilizing the memory space of erased elements. The first approach forms a free list of all erased elements. The second forms a free list of the the first elements in *runs* of contiguous erased elements (ie. "skipblocks" in terms of terminology used in the jump-counting pattern paper). The second can be more efficient, but requires a doubly-linked free list rather than a singly-linked free list - otherwise it becomes an O(N) operation to update links in the skipfield, when a skipblock expands or contracts during erasure or insertion.

The reference implementation currently uses the second approach, using two things to keep track of erased element locations:

a. Metadata for each memory block includes a 'next block with erasures' pointer. The container itself contains a 'blocks with erasures' intrusive list-head pointer. These are used by the container to create an intrusive singly-linked list of memory blocks with erased elements which can be re-used for future insertions.

b. Metadata for each memory block also includes a 'free list head' index number, which gives the index within the memory block of the start node of the last newly-created skipblock. The memory space of this erased element is reinterpret_cast'd as two index numbers, the first ("previous" index) giving the index of the previously erased element, the second ("next" index) giving the next index in the sequence (in this case a unique number because it's pointing to the free list head), and so on - this forms a free list of erased element memory locations which may be re-used.

Previous versions of the reference implementation used a singly-linked free list of erased elements instead of a doubly-linked free list of skipblocks, this was possible with the High complexity jump-counting pattern, but not possible using the Low complexity jump-counting pattern, for various reasons.

One cannot use a stack of pointers to erased elements for this mechanism, as early versions of the reference implementation did, because this can create allocations during erasure, which changes the exception guarantees of erase. One could instead scan all skipfields until an erased location is found, though this would be slow.

In terms of the alternative *boolean + jump-counting skipfield* approach described in the skipfield section above, one could store both the jump-counting data and free list data in any given erased element's memory space, provided of course that elements are aligned to be wide enough to fit both.

Implementation of iterator class

The reference implementation's iterator stores a pointer to the current 'group' struct mentioned above, plus a pointer to the current element and a pointer to its corresponding skipfield node. An alternative approach is to store the group pointer + an index, since the index can indicate both the offset from the memory block for the element, as well as the offset from the start of the skipfield for the skipfield node. However multiple implementations and benchmarks across many processors have shown this to be worse-performing than the separate pointer-based approach, despite the increased memory cost for the iterator class itself.

++ operation is as follows, utilising the reference implementation's Low-complexity jump-counting pattern:

1. Add 1 to the existing element and skipfield pointers.

2. Dereference skipfield pointer to get content of skipfield node, add content of skipfield node to both the skipfield pointer and the element pointer. If the node is erased, its value will be a positive integer indicating the number of nodes until the next non-erased node, if not erased it will be zero.

3. If element pointer is beyond end of element memory block, change group pointer to next group, element pointer to the start of the next group's element memory block, skipfield pointer to the start of the next group's skipfield. Then go back to 2.

-- operation is the same except both step 1 and 2 involve subtraction rather than adding, and step 3 checks to see if element pointer is before the beginning of the element memory blocks and if so relocates to the previous group rather than the next group.

Results of implementation

In practical application the reference implementation is generally faster for insertion and (non-back) erasure than current standard library containers, and generally faster for iteration than any container except vector and deque. See benchmarks here.

V. Technical Specifications

Time complexities for basic operations

In most cases the time complexity of operations results from the fundamental requirements of the container itself, not the implementation. In some cases, such as erase complexity, this can change a little based on the implementation, but the change is minor in practice. For an explanation of any of the following time complexities and how they relate to container requirements, see this entry in the appendix.

- Insert (single): O(1) amortised
- Insert (multiple): O(N) amortised
- Erase (single): O(1) amortised
- Erase (multiple): O(N) amortised for non-trivially-destructible types, for trivially-destructible types between O(1) and O(N) amortised depending on range start/end (approximating O(log n) average)
- std::find: O(n)
- splice: O(1)
- Iterator operators ++ and --: O(1) amortised
- begin()/end(): O(1)
- advance/next/prev: between O(1) and O(n), depending on current iterator location, distance and implementation. Average for reference implementation approximates O(log N).

General specification

Colony meets the requirements of the C++ Container, AllocatorAwareContainer, and ReversibleContainer concepts.

For the most part the syntax and semantics of colony functions are very similar to all std:: c++ libraries. Formal description is as follows:

```
template <class T, class Allocator = std::allocator<T>, typename Skipfield_Type = unsigned short> class colony
```

`T` - the element type. In general T must meet the requirements of Erasable, CopyAssignable and CopyConstructible. However, if emplace is utilized to insert elements into the colony, and no functions which involve copying or moving are utilized, T is only required to meet the requirements of Erasable.
If move-insert is utilized instead of emplace, T must also meet the requirements of MoveConstructible.

`Allocator` - an allocator that is used to acquire memory to store the elements. The type must meet the requirements of Allocator. The behaviour is undefined if `Allocator::value_type` is not the same as T.

`Skipfield_Type` - an unsigned integer type. This type is used to form the skipfield which skips over erased T elements. In terms of the reference implementation, this also acts as a limiting factor to the maximum size of memory blocks, due to the way that the skipfield pattern works (`e.g. unsigned short` is 16-bit on most platforms which constrains the size of individual memory blocks to a maximum of 65535 elements). `unsigned short` has been found to be the optimal type for the current reference implementation. However in the case of small collections (i.e. < 1000 elements) in a memory-constrained environment, it may be useful to reduce the memory usage of the skipfield by reducing the skipfield bit depth to a Uint8 type. The reduced skipfield size may also reduce cache saturation in this case without impacting iteration speed due to the low amount of elements. However whether or not this constitutes a performance advantage is largely situational, so it is best to leave control in the end user's hands.

Basic example of usage (using reference implementation)

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
```

```
{
  plf::colony<int> i_colony;

  // Insert 100 ints:
  for (int i = 0; i != 100; ++i)
  {
    i_colony.insert(i);
  }

  // Erase half of them:
  for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    it = i_colony.erase(it);
  }

  // Total the remaining ints:
  int total = 0;

  for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    total += *it;
  }

  std::cout << "Total: " << total << std::endl;
  std::cin.get();
  return 0;
}
```

Example demonstrating pointer stability

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
{
  plf::colony<int> i_colony;
  plf::colony<int>::iterator it;
  plf::colony<int *> p_colony;
  plf::colony<int *>::iterator p_it;

  // Insert 100 ints to i_colony and pointers to those ints to p_colony:
  for (int i = 0; i != 100; ++i)
  {
    it = i_colony.insert(i);
    p_colony.insert(&(*it));
  }

  // Erase half of the ints:
  for (it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    it = i_colony.erase(it);
  }

  // Erase half of the int pointers:
  for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
  {
    p_it = p_colony.erase(p_it);
  }

  // Total the remaining ints via the pointer colony (pointers will still be valid even after insertions and
  int total = 0;

  for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
  {
    total += *(*p_it);
  }

  std::cout << "Total: " << total << std::endl;

  if (total == 2500)
```

```
  {
    std::cout << "Pointers still valid!" << std::endl;
  }

  std::cin.get();
  return 0;
}
```

Iterator Invalidation

All read-only operations, swap, std::swap, free_unused_memory	Never
clear, sort, reinitialize, operator =	Always
change_block_sizes, change_minimum_block_size, change_maximum_block_size	Only if supplied minimum block size is larger than smallest block in colony, or supplied maximum block size is smaller than largest block in colony.
erase	Only for the erased element. If an iterator is == end() it may be invalidated if the last element in the colony is erased, in some cases (similar to std::deque). If a reverse_iterator is == rend() it may be invalidated if the first element in the colony is erased, in some cases.
insert, emplace	If an iterator is == end() it may be invalidated by a subsequent insert/emplace, in some cases.

Member types

Member type	**Definition**
value_type	T
allocator_type	Allocator
skipfield_type	T_skipfield_type
size_type	std::allocator_traits<Allocator>::size_type
difference_type	std::allocator_traits<Allocator>::difference_type
reference	value_type &
const_reference	const value_type &
pointer	std::allocator_traits<Allocator>::pointer
const_pointer	std::allocator_traits<Allocator>::const_pointer
iterator	BidirectionalIterator
const_iterator	Constant BidirectionalIterator
reverse_iterator	BidirectionalIterator
const_reverse_iterator	Constant BidirectionalIterator

Constructors

standard	colony() explicit colony(allocator_type &alloc)
fill	colony(size_type n, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max(), allocator_type &alloc = allocator_type())

	`explicit colony(size_type n, value_type &element, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max(), allocator_type &alloc = allocator_type())`
range	`template<typename InputIterator> colony(InputIterator &first, InputIterator &last, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max(), allocator_type &alloc = allocator_type())`

copy	`colony(colony &source)`
	
	`colony(colony &source, allocator_type &alloc)`

move	`colony(colony &&source) noexcept`
	
	`colony(colony &&source, allocator_type &alloc)`
	
	Note: postcondition state of source colony is the same as that of an empty colony.

initializer list	`colony(std::initializer_list<value_type> &element_list, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max(), allocator_type &alloc = allocator_type())`

Some constructor usage examples

- `colony<T> a_colony`

  Default constructor - default minimum block size is 8, default maximum block size is std::numeric_limits<Skipfield_type>::max() (typically 65535). You cannot set the block sizes from the constructor in this scenario, but you can call the change_block_sizes() member function after construction has occurred.
  Example: `std::colony<int> int_colony;`

- `colony<T, the_allocator<T> > a_colony(allocator_type &alloc = allocator_type())`

  Default constructor, but using a custom memory allocator e.g. something other than std::allocator.
  Example: `std::colony<int, tbb::allocator<int> > int_colony;`
  Example2:
  ```
  // Using an instance of an allocator as well as its type
  tbb::allocator<int> alloc_instance;
  std::colony<int, tbb::allocator<int> > int_colony(alloc_instance);
  ```

- `colony<T> a_colony(size_type n, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max())`

  Fill constructor with value_type unspecified, so the value_type's default constructor is used. `n` specifies the number of elements to create upon construction. If `n` is larger than `min_block_size`, the size of the blocks created will either be `n` and `max_block_size`, depending on which is smaller. `min_block_size` (i.e. the smallest possible number of elements which can be stored in a colony block) can be defined, as can the `max_block_size`. Setting the block sizes can be a performance advantage if you know in advance roughly how many objects are likely to be stored in your colony long-term - or at least the rough scale of storage. If that case, using this can stop many small initial blocks being allocated.
  Example: `std::colony<int> int_colony(62);`

- `colony<T> a_colony(std::initializer_list<value_type> &element_list, Skipfield_type min_block_size = 8, Skipfield_type max_block_size = std::numeric_limits<Skipfield_type>::max())`

  Using an initialiser list to insert into the colony upon construction.
  Example: `std::initializer_list<int> &el = {3, 5, 2, 1000};`
  `std::colony<int> int_colony(el, 64, 512);`

- `colony<T> a_colony(colony &source)`

  Copy all contents from source colony, removes any empty (erased) element locations in the process. Size of blocks created is either the total size of the source colony, or the maximum block size of the source colony, whichever is the smaller.
  Example: `std::colony<int> int_colony_2(int_colony_1);`

- `colony<T> a_colony(colony &&source)`

  Move all contents from source colony, does not remove any erased element locations or alter any of the source block sizes. Source colony is now empty and can be safely destructed or otherwise used.
  Example: `std::colony<int> int_colony_1(50, 5, 512, 512); // Fill-construct a colony with min and max block sizes set at 512 elements. Fill with 50 instances of int == 5.`
  `std::colony<int> int_colony_2(std::move(int_colony_1)); // Move all data to int_colony_2. All of the above characteristics are now applied to int_colony2.`

Iterators

Iterators are bidirectional but also provide O(1) time complexity >, <, >= and <= operators for convenience (for example, for use in `for` loops when skipping over multiple elements per loop). The O(1) complexity of these operators are achieved by keeping a record of the order of memory blocks in some way (in the reference implementation this is done via assigning a number to each memory block in its metadata), comparing the relative order of the two iterators' memory blocks via this number, then comparing the memory locations of the elements themselves, if they happen to be in the same memory block. The full list of operators for iterator, reverse_iterator, const_iterator and const_reverse_iterator follow:

```
operator *
operator ->
operator ++
operator --
operator =
operator ==
operator !=
operator <
operator >
operator <=
operator >=
base() (reverse_iterator and const_reverse_iterator only)
```

For more information see the member functions list in the appendices.

VI. Acknowledgements

Matt would like to thank: Glen Fernandes and Ion Gaztanaga for restructuring advice, Robert Ramey for documentation advice, various Boost and SG14 members for support, Baptiste Wicht for teaching me how to construct decent benchmarks, Jonathan Wakely, Sean Middleditch, Patrice Roy and Guy Davidson for standards-compliance advice and critiques, support, representation at meetings and bug reports, that guy from Lionhead for annoying me enough to force me to implement the original skipfield pattern, Jon Blow for some initial advice and Mike Acton for some influence. Also Nico Josuttis for doing such an excellent job in terms of explaining the general format of the structure to the committee.

Appendices

Appendix A: Member functions

Insert

single element	`iterator insert (value_type &val)`
fill	`iterator insert (size_type n, value_type &val)`
range	`template <class InputIterator> iterator insert (InputIterator first, InputIterator last)`
move	`iterator insert (value_type&& val)`
initializer list	`iterator insert (std::initializer_list<value_type> il)`

- `iterator insert(value_type &element)`

  Inserts the element supplied to the colony, using the object's copy-constructor. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

  ```
  std::colony<unsigned int> i_colony;
  i_colony.insert(23);
  ```

- `iterator insert(value_type &&element)`

Moves the element supplied to the colony, using the object's move-constructor. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

```
std::string string1 = "Some text";

std::colony<std::string> data_colony;
data_colony.insert(std::move(string1));
```

- `void insert (size_type n, value_type &val)`

  Inserts `n` copies of `val` into the colony. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Example:

  ```
  std::colony<unsigned int> i_colony;
  i_colony.insert(10, 3);
  ```

- `template <class InputIterator> void insert (InputIterator &first, InputIterator &last)`

  Inserts a series of `value_type` elements from an external source into a colony holding the same `value_type` (e.g. int, float, a particular class, etcetera). Stops inserting once it reaches `last`. Example:

  ```
  // Insert all contents of colony2 into colony1:
  colony1.insert(colony2.begin(), colony2.end());
  ```

- `void insert (std::initializer_list<value_type> &il)`

  Copies elements from an initializer list into the colony. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Example:

  ```
  std::initializer_list<int> some_ints = {4, 3, 2, 5};

  std::colony<int> i_colony;
  i_colony.insert(some_ints);
  ```

- `iterator emplace(Arguments &&...parameters)`

  Constructs new element directly within colony. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. "...parameters" are whatever parameters are required by the element's constructor. Example:

  ```
  class simple_class
  {
  private:
  int number;
  public:
  simple_class(int a_number): number (a_number) {};
  };

  std::colony<simple_class> simple_classes;
  simple_classes.emplace(45);
  ```

Erase

single element	`iterator erase(const_iterator it)`
range	`void erase(const_iterator first, const_iterator last)`

- `iterator erase(const_iterator it)`

  Removes the element pointed to by the supplied iterator, from the colony. Returns an iterator pointing to the next non-erased element in the colony (or to end() if no more elements are available). Attempting to erase a previously-erased element results in undefined behaviour (this is checked for via an assert in debug mode). Example:

  ```
  std::colony<unsigned int> data_colony(50);
  std::colony<unsigned int>::iterator an_iterator;
  an_iterator = data_colony.insert(23);
  an_iterator = data_colony.erase(an_iterator);
  ```

- `void erase(const_iterator first, const_iterator last)`

  Erases all elements of a given colony from `first` to the element before the `last` iterator. This function is optimized for multiple consecutive erasures and will always be faster than sequential single-element erase calls in that scenario. Example:

```
std::colony<int> iterator1 = colony1.begin();
colony1.advance(iterator1, 10);
std::colony<int> iterator2 = colony1.begin();
colony1.advance(iterator2, 20);
colony1.erase(iterator1, iterator2);
```

Other functions

- `bool empty()`

  Returns a boolean indicating whether the colony is currently empty of elements.
  Example: `if (object_colony.empty()) return;`

- `size_type size()`

  Returns total number of elements currently stored in container.
  Example: `std::cout << i_colony.size() << std::endl;`

- `size_type max_size()`

  Returns the maximum number of elements that the allocator can store in the container.
  Example: `std::cout << i_colony.max_size() << std::endl;`

- `size_type capacity()`

  Returns total number of elements currently able to be stored in container without expansion.
  Example: `std::cout << i_colony.capacity() << std::endl;`

- `void clear()`

  Empties the colony and removes all elements and blocks.
  Example: `object_colony.clear();`

- `void change_group_sizes(Skipfield_type min_group_size, Skipfield_type max_group_size)`

  Changes the minimum and maximum internal group sizes, in terms of number of elements stored per group. If the colony is not empty and either min_group_size is larger than the smallest group in the colony, or max_group_size is smaller than the largest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new group sizes, invalidating all pointers/iterators/references. If trying to change group sizes with a colony storing a non-copyable/movable type, please use the reinitialize function instead.
  Example: `object_colony.change_group_sizes(1000, 10000);`

- `void change_minimum_group_size(Skipfield_type min_group_size)`

  Changes the minimum internal group size only, in terms of minimum number of elements stored per group. If the colony is not empty and min_group_size is larger than the smallest group in the colony, the colony will be internally move-constructed (if possible) or copy-constructed into a new colony which uses the new minimum group size, invalidating all pointers/iterators/references. If trying to change group sizes with a colony storing a non-copyable/movable type, please use the reinitialize function instead.
  Example: `object_colony.change_minimum_group_size(100);`

- `void change_maximum_group_size(Skipfield_type min_group_size)`

  Changes the maximum internal group size only, in terms of maximum number of elements stored per group. If the colony is not empty and either max_group_size is smaller than the largest group in the colony, the colony will be internally move-constructed (if possible) or copy-constructed into a new colony which uses the new maximum group size, invalidating all pointers/iterators/references. If trying to change group sizes with a colony storing a non-copyable/movable type, please use the reinitialize function instead.
  Example: `object_colony.change_maximum_group_size(1000);`

- `void reinitialize(Skipfield_type min_group_size, Skipfield_type max_group_size)`

  Semantics of this function are the same as "clear(); change_group_sizes(min_group_size, max_group_size);", but without the move/copy-construction code of the change_group_sizes() function - this means it can be used with element types which are non-copy-constructible and non-move-constructible, unlike change_group_sizes().
  Example: `object_colony.reinitialize(1000, 10000);`

- `void swap(colony &source)`

  Swaps the colony's contents with that of `source`.
  Example: `object_colony.swap(other_colony);`

- `void sort();`

  ```
  template <class comparison_function>
  ```

`void sort(comparison_function compare);`

Sort the content of the colony. By default this compares the colony content using a less-than operator, unless the user supplies a comparison function (i.e. same conditions as std::list's sort function). Uses std::sort internally but will use plf::timsort if plf_timsort.h is included in the project before plf_colony.h.
Example:
```
// Sort a colony of integers in ascending order:
int_colony.sort();
// Sort a colony of doubles in descending order:
double_colony.sort(std::greater<double>());
```

- `void splice(colony &source)`

  Transfer all elements from source colony into destination colony without invalidating pointers/iterators to either colony's elements (in other words the destination takes ownership of the source's memory blocks). After the splice, the source colony is empty. Splicing is much faster than range-moving or copying all elements from one colony to another. Colony does not guarantee a particular order of elements after splicing, for performance reasons; the insertion location of source elements in the destination colony is chosen based on the most positive performance outcome for subsequent iterations/insertions. For example if the destination colony is {1, 2, 3, 4} and the source colony is {5, 6, 7, 8} the destination colony post-splice could be {1, 2, 3, 4, 5, 6, 7, 8} or {5, 6, 7, 8, 1, 2, 3, 4}, depending on internal state of both colonies and prior insertions/erasures.

  Note: If the minimum block size of the source is smaller than the destination, the destination will change its minimum block size to match the source. The same applies for maximum block sizes (if source's is larger, the destination will adjust its size).
  Example:
  ```
  // Splice two colonies of integers together:
  colony<int> colony1 = {1, 2, 3, 4}, colony2 = {5, 6, 7, 8};
  colony1.splice(colony2);
  ```

- `colony & operator = (colony &source)`

  Copy the elements from another colony to this colony, clearing this colony of existing elements first.
  Example:
  ```
  // Manually swap data_colony1 and data_colony2 in C++03
  data_colony3 = data_colony1;
  data_colony1 = data_colony2;
  data_colony2 = data_colony3;
  ```

- `colony & operator = (colony &&source)`

  Move the elements from another colony to this colony, clearing this colony of existing elements first. Source colony is now empty and in a valid state (same as a new colony without any insertions), can be safely destructed or used in any regular way without problems.
  Example:
  ```
  // Manually swap data_colony1 and data_colony2
  data_colony3 = std::move(data_colony1);
  data_colony1 = std::move(data_colony2);
  data_colony2 = std::move(data_colony3);
  ```

- `bool operator == (colony &source)`

  Compare contents of another colony to this colony. Returns a boolean as to whether they are equal.
  Example: `if (object_colony == object_colony2) return;`

- `bool operator != (colony &source)`

  Compare contents of another colony to this colony. Returns a boolean as to whether they are not equal.
  Example: `if (object_colony != object_colony2) return;`

- `iterator begin(), iterator end(), const_iterator cbegin(), const_iterator cend()`

  Return iterators pointing to, respectively, the first element of the colony and the element one-past the end of the colony (as per standard STL guidelines).

- `reverse_iterator rbegin(), reverse_iterator rend(), const_reverse_iterator crbegin(), const_reverse_iterator crend()`

  Return reverse iterators pointing to, respectively, the last element of the colony and the element one-before the first element of the colony (as per standard STL guidelines).

- `iterator get_iterator_from_pointer(element_pointer_type the_pointer)`

  Getting a pointer from an iterator is simple - simply dereference it then grab the address i.e. `"&(*the_iterator);"`. Getting an iterator from a pointer is typically not so simple. This function enables the user to do exactly that. This is expected to be useful in the use-case where external containers are storing pointers to colony elements instead of iterators (as iterators for colonies have 3 times the size of an element pointer) and the program wants to erase the element being pointed to or possibly change the element being pointed to. Converting a pointer to an iterator using this method and then erasing, is about 20% slower on average than erasing when you already have the iterator. This is less dramatic than it sounds, as it is still faster than other std:: container erasure times. However this is generally a slower, lookup-based operation. If the lookup doesn't find a non-erased element based on that pointer, it returns `end()`. Otherwise it returns an iterator pointing to the element in question. Example:

  ```
  std::colony<a_struct> data_colony;
  std::colony<a_struct>::iterator an_iterator;
  ```

```
a_struct struct_instance;
an_iterator = data_colony.insert(struct_instance);
a_struct *struct_pointer = &(*an_iterator);
iterator another_iterator = data_colony.get_iterator_from_pointer(struct_pointer);
if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
```

- `size_type get_index_from_iterator(iterator/const_iterator &the_iterator)` **(slow)**

  While colony is a container with unordered insertion (and is therefore unordered), it still has a (transitory) order which changes upon any erasure or insertion. *Temporary* index numbers are therefore obtainable. These can be useful, for example, when creating a save file in a computer game, where certain elements in a container may need to be re-linked to other elements in other container upon reloading the save file. Example:

  ```
  std::colony<a_struct> data_colony;
  std::colony<a_struct>::iterator an_iterator;
  a_struct struct_instance;
  data_colony.insert(struct_instance);
  data_colony.insert(struct_instance);
  an_iterator = data_colony.insert(struct_instance);
  unsigned int index = data_colony.get_index_from_iterator(an_iterator);
  if (index == 2) std::cout << "Index is correct" << std::endl;
  ```

- `size_type get_index_from_reverse_iterator(reverse_iterator/const_reverse_iterator &the_iterator)` **(slow)**

  The same as get_index_from_iterator, but for reverse_iterators and const_reverse_iterators. Index is from front of colony (same as iterator), not back of colony. Example:

  ```
  std::colony<a_struct> data_colony;
  std::colony<a_struct>::reverse_iterator r_iterator;
  a_struct struct_instance;
  data_colony.insert(struct_instance);
  data_colony.insert(struct_instance);
  r_iterator = data_colony.rend();
  unsigned int index = data_colony.get_index_from_reverse_iterator(r_iterator);
  if (index == 1) std::cout << "Index is correct" << std::endl;
  ```

- `iterator get_iterator_from_index(size_type index)` **(slow)**

  As described above, there may be situations where obtaining iterators to specific elements based on an index can be useful, for example, when reloading save files. This function is basically a shorthand to avoid typing "`iterator it = colony.begin(); colony.advance(it, 50);`". Example:

  ```
  std::colony<a_struct> data_colony;
  std::colony<a_struct>::iterator an_iterator;
  a_struct struct_instance;
  data_colony.insert(struct_instance);
  data_colony.insert(struct_instance);
  iterator an_iterator = data_colony.insert(struct_instance);
  iterator another_iterator = data_colony.get_iterator_from_index(2);
  if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
  ```

- `allocator_type get_allocator()`

  Returns a copy of the allocator used by the colony instance.

Non-member functions

- `void swap(colony &A, source &B)`

  Swaps colony A's contents with that of colony B (assumes both colonies have same element type, allocator type, etc).
  Example: `swap(object_colony, other_colony);`

- `template <iterator_type> void advance(iterator_type iterator, distance_type distance)`

  Increments/decrements the iterator supplied by the positive or negative amount indicated by *distance*. Speed of incrementing will almost always be faster than using the ++ operator on the iterator for increments greater than 1. In some cases it may approximate O(1). The iterator_type can be an iterator, const_iterator, reverse_iterator or const_reverse_iterator.
  Example: `colony<int>::iterator it = i_colony.begin();`
  `i_colony.advance(it, 20);`

- `template <iterator_type> iterator_type next(iterator_type &iterator, distance_type distance)`

  Creates a copy of the iterator supplied, then increments/decrements this iterator by the positive or negative amount indicated by *distance*.
  Example: `colony<int>::iterator it = i_colony.next(i_colony.begin(), 20);`

- `template <iterator_type> iterator_type prev(iterator_type &iterator, distance_type distance)`

  Creates a copy of the iterator supplied, then decrements/increments this iterator by the positive or negative amount indicated by *distance*.
  Example: `colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20);`

- `template <iterator_type> difference_type distance( iterator_type &first, iterator_type &last)`

  Measures the distance between two iterators, returning the result, which will be negative if the second iterator supplied is before the first iterator supplied in terms of its location in the colony.
  Example:
```
colony<int>::iterator it = i_colony.next(i_colony.begin(), 20);
colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20);
std::cout "Distance: " i_colony.distance(it, it2) std::endl;
```

Note: the four immediately above are member functions in the reference implementation as a workaround for an unfixed bug in MSVC2013.

Appendix B - reference implementation benchmarks

Benchmark results for the colony v5 reference implementation under GCC 8.1 x64 on an Intel Xeon E3-1241 (Haswell) are here.

Old benchmark results for an earlier version of colony under MSVC 2015 update 3, on an Intel Xeon E3-1241 (Haswell) are here. There is no commentary for the MSVC results.

Appendix C - Frequently Asked Questions

1. **Where is it worth using a colony in place of other std:: containers?**

   As mentioned, it is worthwhile for performance reasons in situations where the order of container elements is not important and:

   a. Insertion order is unimportant

   b. Insertions and erasures to the container occur frequently in performance-critical code, ***and***

   c. Links to non-erased container elements may not be invalidated by insertion or erasure.

   Under these circumstances a colony will generally out-perform other std:: containers. In addition, because it never invalidates pointer references to container elements (except when the element being pointed to has been previously erased) it may make many programming tasks involving inter-relating structures in an object-oriented or modular environment much faster, and could be considered in those circumstances.

2. **What are some examples of situations where a colony might improve performance?**

   Some ideal situations to use a colony: cellular/atomic simulation, persistent octtrees/quadtrees, game entities or destructible-objects in a video game, particle physics, anywhere where objects are being created and destroyed continuously. Also, anywhere where a vector of pointers to dynamically-allocated objects or a std::list would typically end up being used in order to preserve pointer stability but where order is unimportant.

3. **Is it similar to a deque?**

   A deque is reasonably dissimilar to a colony - being a double-ended queue, it requires a different internal framework. In addition, being a random-access container, having a growth factor for memory blocks in a deque is problematic (not impossible though). A deque and colony have no comparable performance characteristics except for insertion (assuming a good deque implementation). Deque erasure performance varies wildly depending on the implementation, but is generally similar to vector erasure performance. A deque invalidates pointers to subsequent container elements when erasing elements, which a colony does not, and is ordered.

4. **What are the thread-safe guarantees?**

   Unlike a std::vector, a colony can be read from and inserted into at the same time (assuming different locations for read and

write), however it cannot be iterated over and written to at the same time. If we look at a (non-concurrent implementation of) std::vector's thread-safe matrix to see which basic operations can occur at the same time, it reads as follows (please note push_back() is the same as insertion in this regard):

std::vector	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	No
Erasure	No	No	No	No
Iteration	No	No	Yes	Yes
Read	No	No	Yes	Yes

In other words, multiple reads and iterations over iterators can happen simultaneously, but the potential reallocation and pointer/iterator invalidation caused by insertion/push_back and erasure means those operations cannot occur at the same time as anything else.

Colony on the other hand does not invalidate pointers/iterators to non-erased elements during insertion and erasure, resulting in the following matrix:

colony	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	Yes
Erasure	No	No	No	Mostly*
Iteration	No	No	Yes	Yes
Read	Yes	Mostly*	Yes	Yes

* Erasures will not invalidate iterators unless the iterator points to the erased element.

In other words, reads may occur at the same time as insertions and erasures (provided that the element being erased is not the element being read), multiple reads and iterations may occur at the same time, but iterations may not occur at the same time as an erasure or insertion, as either of these may change the state of the skipfield which is being iterated over. Note that iterators pointing to end() may be invalidated by insertion.

So, colony could be considered more inherently thread-safe than a (non-concurrent implementation of) std::vector, but still has some areas which would require mutexes or atomics to navigate in a multithreaded environment.

5. **Any pitfalls to watch out for?**

   Because erased-element memory locations may be reused by `insert()` and `emplace()`, insertion position is essentially random unless no erasures have been made, or an equal number of erasures and insertions have been made.

6. **What is the purpose of limiting memory block minimum and maximum sizes?**

   One reason might be to ensure that memory blocks match a certain processor's cache or memory pathway sizes. Another reason to do this is that it is slightly slower to obtain an erased-element location from the list of groups-with-erasures (subsequently utilising that group's free list of erased locations) and to reuse that space than to insert a new element to the back of the colony (the default behaviour when there are no previously-erased elements). If there are any erased elements in the colony, the colony will recycle those memory locations, unless the entire block is empty, at which point it is freed to memory.

   So if a block size is large, and many erasures occur but the block is not completely emptied, iterative performance might suffer due to large memory gaps between any two non-erased elements and subsequent drop in data locality and cache performance. In that scenario you may want to experiment with benchmarking and limiting the minimum/maximum sizes of the blocks, such that memory blocks are freed earlier and find the optimal size for the given use case.

7. **What is colony's Abstract Data Type (ADT)?**

   Though I am happy to be proven wrong I suspect colonies/bucket arrays are their own abstract data type. Some have suggested it's ADT is of type bag, I would somewhat dispute this as it does not have typical bag functionality such as searching based on value (you can use std::find but it's o(n)) and adding this functionality would slow down other performance characteristics. Multisets/bags are also not sortable (by means other than automatically by key value). Colony does not utilize key values, is

sortable, and does not provide the sort of functionality frequently associated with a bag (e.g. counting the number of times a specific value occurs).

8. **Why must blocks be removed when empty?**

   Two reasons:

   a. Standards compliance: if blocks aren't removed then `++` and `--` iterator operations become undefined in terms of time complexity, making them non-compliant with the C++ standard. At the moment they are O(1) amortised, typically one update for both skipfield and element pointers, but two if a skipfield jump takes the iterator beyond the bounds of the current block and into the next block. But if empty blocks are allowed, there could be anywhere between 1 and `std::numeric_limits<size_type>::max` empty blocks between the current element and the next. Essentially you get the same scenario as you do when iterating over a boolean skipfield. It would be possible to move these to the back of the colony as trailing blocks, or house them in a separate list or vector for future usage, but this may create performance issues if any of the blocks are not at their maximum size (see below).

   b. Performance: iterating over empty blocks is slower than them not being present, of course - but also if you have to allow for empty blocks while iterating, then you have to include a while loop in every iteration operation, which increases cache misses and code size. The strategy of removing blocks when they become empty also statistically removes (assuming randomised erasure patterns) smaller blocks from the colony before larger blocks, which has a net result of improving iteration, because with a larger block, more iterations within the block can occur before the end-of-block condition is reached and a jump to the next block (and subsequent cache miss) occurs. Lastly, pushing to the back of a colony, provided there is still space and no new block needs to be allocated, will be faster than recycling memory locations as each subsequent insertion occurs in a subsequent memory location (which is cache-friendlier) and also less computational work is necessary. If a block is removed its recyclable memory locations are also of course removed, hence subsequent insertions are more likely to be pushed to the back of the colony.

9. **Why not preserve empty memory blocks for future use, in a separate list or vector instead of freeing them to the OS, or leave this decision undefined by the specification?**

   The default scenario, for reasons of predictability, should be to free the memory block rather than making this undefined. If a scenario calls for retaining memory blocks instead of deallocating them, this should be left to an allocator to manage. Otherwise you get unpredictable memory behaviour across implementations, and this is one of the things that SG14 members have complained about time-and-time again, the lack of predictable behaviour across standard library implementations. Ameliorating this unpredictability is best in my view.

10. **Memory block sizes - what are they based on, how do they expand, etc**

    In the reference implementation memory block sizes start from either the default minimum size (8 elements, larger if the type stored is small) or an amount defined by the programmer (with a minimum of 3 elements). Subsequent block sizes then increase the *total capacity* of the colony by a factor of 2 (so, 1st block 8 elements, 2nd 8 elements, 3rd 16 elements, 4th 32 elements etcetera) until the maximum block size is reached. The default maximum block size is the maximum possible number that the skipfield bitdepth is capable of representing (std::numeric_limits<skipfield_type>::max()). By default the skipfield bitdepth is 16 so the maximum size of a block is 65535 elements.

    However the skipfield bitdepth is also a template parameter which can be set to any unsigned integer - unsigned char, unsigned int, Uint_64, etc. Unsigned short (guaranteed to be at least 16 bit, equivalent to C++11's uint_least16_t type) was found to have the best performance in real-world testing due to the balance between memory contiguousness, memory waste and the number of allocations.

11. **Can a colony be used with SIMD instructions?**

    No and yes. Yes if you're careful, no if you're not.
    On platforms which support scatter and gather operations via hardware (e.g. AVX512) you can use colony with SIMD as much as you want, using gather to load elements from disparate or sequential locations, directly into a SIMD register, in parallel. Then use scatter to push the post-SIMD-process values elsewhere after. On platforms which do not support this in hardware, you would need to manually implement a scalar gather-and-scatter operation which may be significantly slower.

    In situations where gather and scatter operations are too expensive, which require elements to be contiguous in memory for SIMD processing, this is more complicated. When you have a bunch of erasures in a colony, there's no guarantee that your objects will be contiguous in memory, even though they are sequential during iteration. Some of them may also be in different memory blocks to each other. In these situations if you want to use SIMD with colony, you must do the following:

    - Set your minimum and maximum group sizes to multiples of the width of your SIMD instruction. If it supports 8 elements at once, set the group sizes to multiples of 8.
    - Either never erase from the colony, or:
      1. Shrink-to-fit after you erase (will invalidate all pointers to elements within the colony).

2. Only erase from the back or front of the colony, and only erase elements in multiples of the width of your SIMD instruction e.g. 8 consecutive elements at once. This will ensure that the end-of-memory-block boundaries line up with the width of the SIMD instruction, provided you've set your min/max block sizes as above.

Generally if you want to use SIMD without gather/scatter, it's probably preferable to use a vector or an array.

Appendix D - Specific responses to previous committee feedback

1. **"Why not 'bag'? Colony is too selective a name."**

   'bag' is problematic partially because it has been synonymous with a multiset (and colony is not one of those) in both computer science and mathematics since the 1970s, and partially because it's a bit vague - it doesn't describe how the container works. However I accept that it is a familiar name and describes a similar territory, for most programmers and will accept that as a id if needed. 'colony' is an intuitive name if you understand the container, and allows for easy conveyance of how it functions internally (colony = human colony/ant colony etc, memory blocks = houses, elements = people/ants in the houses who come and go). The claim that the use of the word is selective in terms of its meaning, is also true for vector, set, 'bag', and many other C++ names.

2. **"Unordered and no associative lookup, so this only supports use cases where you're going to do something to every element."**

   As noted the container was originally designed for highly object-oriented situations where you have many elements in different containers linking to many other elements in other containers. This linking can be done with pointers or iterators in colony (insert returns an iterator which can be dereferenced to get a pointer, pointers can be converted into iterators with the supplied functions (for erase etc)) and because pointers/iterators stay stable regardless of insertion/erasure, this usage is unproblematic. You could say the pointer is equivalent to a key in this case (but without the overhead). That is the first access pattern, the second is straight iteration over the container, as you say. Secondly, the container does have (typically better than O(n)) advance/next/prev implementations, so multiple elements can be skipped.

3. **"Do we really need the skipfield_type template argument?"**

   This argument currently promotes use of the container in heavily memory-constrained environments, and in high-performance small-N collections (where the type of the skipfield can be reduced to 8 bits without having a negative effect on maximum block sizes and subsequent iteration speed). See more explanation in V. Technical Specifications. Unfortunately this parameter also means `operator =` and some other functions won't work between colonies of the same type but differing skipfield types. Further, the template argument is chiefly relevant to the use of the skipfield patterns utilized in the reference implementations, and there may be better techniques.

   However, the parameter can always be ignored in an implementation. Retaining it, even if significantly advanced structures are discovered for skipping elements, harms nothing and can be deprecated if necessary. At this point in time I do not personally see many alternatives to the two skipfield patterns which have been used in the references implementations, both of which benefit from having this optional parameter. Please note, that is not the same as saying there are no alternatives, just ones never thought of yet. This is something I am flexible on, as a singular skipfield type will cover the majority of scenarios.

   Research into this area has determined that there is only really an advantage to using unsigned char for the skipfield type if the number of elements is under 1000, and not in all scenarios. So whether or not this constitutes a performance gain is largely scenario-dependent, certainly it always constitutes a memory usage reduction but the relative effect of this depends on the size of your stored type.

4. **"Prove this is not an allocator"**

   I'm not really sure how to answer this, as I don't see the resemblance, unless you count maps, vectors etc as being allocators also. The only aspect of it which resembles what an allocator might do, is the memory re-use mechanism. It would be impossible for an allocator to perform a similar function while still allowing the container to iterate over the data linearly in memory, preserving locality, in the manner described in this document.

5. **"If this is for games, won't game devs just write their own versions for specific types in order to get a 1% speed increase anyway?"**

   This is true for many/most AAA game companies who are on the bleeding edge, but they also do this for vector etc, so they aren't the target audience of std:: for the most part; sub-AAA game companies are more likely to use third party/pre-existing tools. As mentioned earlier, this structure (bucket-array-like) crops up in many, many fields, not just game dev. So the target

audience is probably everyone other than AAA gaming, but even then, it facilitates communication across fields and companies as to this type of container, giving it a standardised name and understanding.

6. **"Is there active research in this problem space? Is it likely to change in future?"**

   The only current analysis has been around the question of whether it's possible for this specification to fail to allow for a better implementation in future. This is unlikely given the container's requirements and how this impacts on implementation. Bucket arrays have been around since the 1990s, there's been no significant innovation in them until now. I've been researching/working on colony since early 2015, and while I can't say for sure that a better implementation might not be possible, I am confident that no change should be necessary to the specification to allow for future implementations, if it is done correctly.

   The requirement of allowing no reallocations upon insertion or erasure, truncates possible implementation strategies significantly. Memory blocks have to be independently allocated so that they can be removed (when empty) without triggering reallocation of subsequent elements. There's limited numbers of ways to do that and keep track of the memory blocks at the same time. Erased element locations must be recorded (for future re-use by insertion) in a way that doesn't create allocations upon erasure, and there's limited numbers of ways to do this also. Multiple consecutive erased elements have to be skipped in O(1) time, and again there's limits to how many ways you can do that. That covers the three core aspects upon which this specification is based. See IV. Design Decisions for the various ways these aspects can be designed.

   Skipfield update time complexity should, I think, be left implementation-defined, as defining time complexity may obviate better solutions which are faster but are not necessarily O(1). Skipfield updates occur during erasure, insertion, splicing, sorting and container copying. I have looked into alternatives to a 1-node-per-element skipfield, such as a compressed skipfield (a series of numbers denoting alternating lengths of non-erased/erased elements), but all the possible implementations I can think of either involve resizing of an array on-the-fly (which doesn't work well with low latency) and/or slowing down iteration time significantly.

Appendix E - Typical game engine requirements

Here are some more specific requirements with regards to game engines, verified by game developers within SG14:

a. Elements within data collections refer to elements within other data collections (through a variety of methods - indices, pointers, etc). These references must stay valid throughout the course of the game/level. Any container which causes pointer or index invalidation creates difficulties or necessitates workarounds.

b. Order is unimportant for the most part. The majority of data is simply iterated over, transformed, referred to and utilized with no regard to order.

c. Erasing or otherwise "deactivating" objects occurs frequently in performance-critical code. For this reason methods of erasure which create strong performance penalties are avoided.

d. Inserting new objects in performance-critical code (during gameplay) is also common - for example, a tree drops leaves, or a player spawns in an online multiplayer game.

e. It is not always clear in advance how many elements there will be in a container at the beginning of development, or at the beginning of a level during play. Genericized game engines in particular have to adapt to considerably different user requirements and scopes. For this reason extensible containers which can expand and contract in realtime are necessary.

f. Due to the effects of cache on performance, memory storage which is more-or-less contiguous is preferred.

g. Memory waste is avoided.

std::vector in its default state does not meet these requirements due to:

1. Poor (non-fill) singular insertion performance (regardless of insertion position) due to the need for reallocation upon reaching capacity
2. Insert invalidates pointers/iterators to all elements
3. Erase invalidates pointers/iterators/indexes to all elements after the erased element

Game developers therefore either develop custom solutions for each scenario or implement workarounds for vector. The most common workarounds are most likely the following or derivatives:

1. Using a boolean flag or similar to indicate the inactivity of an object (as opposed to actually erasing from the vector). Elements flagged as inactive are skipped during iteration.

   Advantages: Fast "deactivation". Easy to manage in multi-access environments.
   Disadvantages: Can be slower to iterate due to branching.

2. Using a vector of data and a secondary vector of indexes. When erasing, the erasure occurs only in the vector of indexes, not the vector of data. When iterating it iterates over the vector of indexes and accesses the data from the vector of data via the remaining indexes.

Advantages: Fast iteration.
Disadvantages: Erasure still incurs some reallocation cost which can increase jitter.

3. Combining a swap-and-pop approach to erasure with some form of dereferenced lookup system to enable contiguous element iteration (sometimes called a 'packed array': http://bitsquid.blogspot.ca/2011/09/managing-decoupling-part-4-id-lookup.html).
   Advantages: Iteration is at standard vector speed.
   Disadvantages: Erasure will be slow if objects are large and/or non-trivially copyable, thereby making swap costs large. All link-based access to elements incur additional costs due to the dereferencing system.

Colony brings a more generic solution to these contexts. While some developers, particularly AAA developers, will almost always develop a custom solution for specific use-cases within their engine, I believe most sub-AAA and indie developers are more likely to rely on third party solutions. Regardless, standardising the container will allow for greater cross-discipline communication.

Appendix F - Time complexity requirement explanations

Insert (single): O(1) amortised

One of the requirements of colony is that pointers to non-erased elements stay valid regardless of insertion/erasure within the container. For this reason the container must use multiple memory blocks. If a single memory block were used, like in a std::vector, reallocation of elements would occur when the container expanded (and the elements were copied to a larger memory block). Hence, colony will insert into existing memory blocks when able, and create a new memory block when all existing memory blocks are full. Because colony (by default) has a growth pattern for new memory blocks, the number of insertions before a new memory block is required increases as more insertions take place. Hence the insertion time complexity is O(1) amortised.

Insert (multiple): O(N) amortised

This follows the same principle - there will occasionally be a need to create a new memory block, but since this is infrequent it is O(N) amortised.

Erase (single): O(1) amortised

Generally speaking erasure is a simple matter of destructing the element in question and updating the skipfield. When using the Low-complexity jump-counting pattern this skipfield update is always O(1). However, when a memory block becomes empty of non-erased elements it must be freed to the OS (or stored for future insertions, depending on implementation) and removed from the colony's sequence of memory blocks. It it was not, we would end up with non-O(1) amortised iteration, since there would be no way to predict how many empty memory blocks there would be between the current memory block being iterated over, and the next memory block with non-erased (active) elements in it. Since removal of memory blocks is infrequent the time complexity becomes O(1) amortised.

Erase (multiple): O(N) amortised for non-trivially-destructible types, for trivially-destructible types between O(1) and O(N) amortised depending on range start/end (approximating O(log n) average)

In this case, where the element is non-trivially destructible, the time complexity is O(N) amortised, with infrequent deallocation necessary from the removal of an empty memory block as noted above. However where the elements are trivially-destructible, if the range spans an entire memory block at any point, that block and it's skipfield can simply be removed without doing any individual writes to it's skipfield or individual destruction of elements, potentially making this a O(1) operation.

In addition (when dealing with trivially-destructible types) for those memory blocks where only a portion of elements are erased by the range, if no prior erasures have occurred in that memory block you can erase that range in O(1) time, as there will be no need to check within the range for previously erased elements, and the Low-complexity jump-counting pattern only requires two skipfield writes to indicate a range of skipped nodes. The reason you would need to check for previously erased elements within that portion's range is so you can update the metadata for that memory block to accurately reflect how many non-erased elements remain within the block. If that metadata is not present, there is no way to ascertain when a memory block is empty of non-erased elements and hence needs to be removed from the colony. The reasoning for why empty memory blocks must be removed is included in the Erase(single) section, above.

However in most cases the erase range will not perfectly match the size of all memory blocks, and with typical usage of a colony there is usually some prior erasures in most memory blocks. So for example when dealing with a colony of a trivially-destructible type, you might end up with a tail portion of the first memory block in the erasure range being erased in O(N) time, the second and intermediary memory block being completely erased and freed in O(1) time, and only a small front portion of the third and final memory block in the range being erased in O(N) time. Hence the time complexity for trivially-destructible elements approximates O(log n) on average, being between O(1) and O(N) depending on the start and end of the erasure range.

std::find: O(n)

This relies on basic iteration so is O(N).

splice: O(1)

Colony only does full-container splicing, not partial-container splicing (use range-insert with std::move to achieve the latter, albiet with the loss of pointer validity to the moved range). When splicing the memory blocks from the source colony are transferred to the destination colony without processing the individual elements or skipfields, inserted after the destination's final block. If there are unused element memory spaces at the back of the destination container (ie. the final memory block is not full), the skipfield nodes corresponding to those empty spaces must be altered to indicate that these are skipped elements. Again when using the Low-complexity jump-counting pattern for the skipfield this is also a O(1) operation, hence the overall operation is O(1).

Iterator operators ++ and --: O(1) amortised

Generally the time complexity is O(1), as the skipfield pattern used must allow for skipping of multiple erased elements. However every so often iteration will involve a transistion to the next/previous memory block in the colony's sequence of blocks, depending on whether we are doing ++ or --. At this point a read of the next/previous memory block's corresponding skipfield is necessary, in case the first/last element(s) in that memory block are erased and hence skipped. So for every block transition, 2 reads of the skipfield are necessary instead of 1. Hence the time complexity is O(1) amortised.

Skipfields must be per-block and independent between memory blocks, as otherwise you would end up with a vector for a skipfield, which would need a range erased every time a memory block was removed from the colony (see notes under Erase above), and reallocation to larger skipfield memory block when the colony expanded. Both of these procedures carry reallocation costs, meaning you could have thousands of skipfield nodes needing to be reallocated based on a single erasure (from within a memory block which only had one non-erased element left and hence would need to be removed from the colony). This is unacceptable latency for any field involving high timing sensitivity (all of SG14).

begin()/end(): O(1)

For any implementation these should generally be stored as member variables and so returning them is O(1).

advance/next/prev: between O(1) and O(n), depending on current iterator location, distance and implementation. Average for reference implementation approximates O(log N).

The reasoning for this is similar to that of Erase(multiple), above. Memory blocks which are completely traversed by the increment/decrement distance can be skipped in O(1) time by reading the metadata for the block which details the number of non-erased elements in the block. If the current iterator location is not at the very beginning of a memory block, then the distance within that block will need to be processed in O(N) time, unless no erasures have occurred within that block, in which case the skipfield does not need to be checked and traversal is simply a matter of pointer arithmetic and checking the distance between the current iterator position and the end of the memory block, which is an O(1) operation.

Similarly for the final block involved in the traversal (if the traversal spans multiple memory blocks, and the traversals remaining do not exactly match the number of elements in the final block, in which case it is also skipped!), if no erasures have occurred in that memory block then pointer arithmetic can be used and the procedure becomes O(1). Otherwise it is O(N) (++ iterating over the skipfield). We can therefore say that the overall time complexity for these operations might approximate O(log N), while being between O(1) and O(N) amortised respectively.

Since the traversal depends in part of the skipfield, and on what metadata is stored about the memory blocks, implementation can affect the assumptions above. This analysis is based upon the reference implementation, which allows for these possible O(1) sub-operations.

Appendix G - Paper revision history

- R10: Additional information about time complexity requirements added to appendix, some minor corrections to time complexity info. The 'bentley pattern' (this was always a temporary name) is renamed to the more astute 'low-complexity jump-counting pattern'. Likewise the 'advanced jump-counting skipfield' is renamed to the 'high-complexity jump-counting pattern' - for reasoning behind this go here. Both refer to time complexity of operations, as opposed to algorithmic complexity. Additional alternative approach added to Design Decisions under skipfield information. Some other corrections.
- R9: Link to Bentley pattern paper added, and is spellchecked now.
- R8: Correction to SIMD info. Correction to structure (missing appendices title, member functions and technical specification were conjoined, acknowledgments section had mysteriously gone missing since an earlier version, now restored and updated).

Update intro. HTML corrections.

- R7: Minor changes to member functions.
- R6: Re-write. Reserve() and shrink_to_fit() removed from specification.
- R5: Additional note for reserve, re-write of introduction.
- R4: Addition of revision history and review feedback appendices. General rewording. Update of benchmarks to v4 of colony, using max 1000000 N for most benchmarks, and using GCC 7.1 as compiler on a Haswell-core machine. Previous benchmarks also still available at external links. Expansion of initial metaphorical explanation. Cutting of some dead wood. Addition of some more dead wood. Reversion to HTML, benchmarks moved to external URL, based on feedback. Change of font to Times New Roman based on looking at what other papers were using, though I did briefly consider Comic Sans. Change to insert specifications.
- R3: Jonathan Wakely's extensive technical critique has been actioned on, in both documentation and the reference implementation. "Be clearer about what operations this supports, early in the paper." - done (V. Technical Specifications). "Be clear about the O() time of each operation, early in the paper." - done for main operations, see V. Technical Specifications. Responses to some other feedbacks included in the foreword.
- R2: Rewording.